Machine Learning Notes
Preliminary Introduction to Deep Learning

Vitor N. Cabral de Oliveira
vnco@cin.ufpe.br

December 2024

This notebook serves as a comprehensive repository for my Machine Learning studies, with a primary focus on Deep Learning. Key topics covered include:

- Foundations of statistical learning.
- Artificial Neural Networks (ANNs) – Fundamentals and architectures with mathematical explanation.

Contents

Preliminary Foundations of Statistical Learning

Before diving into neural network architectures, it is essential to establish the fundamental concepts of statistical learning theory. This framework formalizes the problem of learning from data and provides the theoretical basis for algorithms such as the perceptron and multilayer networks.

The Learning Problem

In supervised learning, we are given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where each $\mathbf{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d$ is an input vector (feature vector) and $y_i \in \mathcal{Y}$ is the corresponding target output. The nature of $\mathcal{Y}$ defines the task:

- **Classification:** $\mathcal{Y} = \{1, \ldots, K\}$ (finite set of class labels).
- **Regression:** $\mathcal{Y} = \mathbb{R}$ (real-valued output).

The fundamental assumption is that the training examples are drawn independently and identically distributed (i.i.d.) from an unknown joint probability distribution $P(\mathbf{x}, y)$ over $\mathcal{X} \times \mathcal{Y}$.

Hypothesis Space and Loss Functions

A learning algorithm selects a hypothesis $h : \mathcal{X} \to \mathcal{Y}$ from a predefined hypothesis space $\mathcal{H}$. The quality of a hypothesis is measured by a **loss function** $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$, where $\ell(h(\mathbf{x}), y)$ quantifies the cost of predicting $h(\mathbf{x})$ when the true target is y. Common loss functions include:

- **0–1 loss (classification):** $\ell(h(\mathbf{x}), y) = \mathbf{1}_{h(\mathbf{x}) \neq y}$.
- **Squared loss (regression):** $\ell(h(\mathbf{x}), y) = (y - h(\mathbf{x}))^2$.
- **Cross-entropy loss (classification with probabilities):** $\ell(\hat{\mathbf{p}}, y) = -\sum_{k=1}^K \mathbf{1}_{y=k} \log \hat{p}_k$.

The ultimate goal is to minimize the **expected risk** (also called true risk or generalization error):

$$R(h) = \mathbb{E}_{(\mathbf{x},y)\sim P} \left[\ell(h(\mathbf{x}), y) \right]. \tag{1}$$

Since P is unknown, we cannot directly compute $R(h)$. Instead, we minimize the **empirical risk** over the training set:

$$\widehat{R}_{\mathcal{D}}(h) = \frac{1}{N} \sum_{i=1}^N \ell(h(\mathbf{x}_i), y_i). \tag{2}$$

Empirical Risk Minimization (ERM)

The principle of empirical risk minimization (ERM) [1] selects the hypothesis that minimizes the empirical risk:

$$h_{\text{ERM}} = \arg\min_{h \in \mathcal{H}} \widehat{R}_{\mathcal{D}}(h). \tag{3}$$

For finite $\mathcal{H}$, uniform convergence bounds (e.g., Hoeffding's inequality together with union bound) guarantee that with high probability, $R(h_{\text{ERM}})$ is close to the minimal possible risk $\min_{h \in \mathcal{H}} R(h)$, provided the sample size N is sufficiently large relative to the complexity of $\mathcal{H}$. This interplay between approximation error (bias) and estimation error (variance) is captured by the **bias–variance decomposition** (for squared loss):

$$\mathbb{E}_{\mathcal{D}}\left[R(h_{\mathcal{D}})\right] = \underbrace{R(h^*)}_{\text{irreducible error}} + \underbrace{\left(\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}] - h^*\right)^2}_{\text{bias}^2} + \underbrace{\mathbb{V}_{\mathcal{D}}[h_{\mathcal{D}}]}_{\text{variance}}. \tag{4}$$

where h^* is the Bayes-optimal hypothesis. Larger hypothesis spaces reduce bias but increase variance, leading to the classic **bias–variance tradeoff**.

Stochastic Gradient Descent as an Optimization Method

Most modern machine learning models, including neural networks, are trained by minimizing the empirical risk using gradient-based optimization. Because the empirical risk is an average over N terms,

$$\widehat{R}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(\mathbf{w}),$$

the full gradient $\nabla \widehat{R}_{\mathcal{D}}(\mathbf{w})$ requires a pass through the entire dataset, which is computationally expensive for large N. The stochastic gradient descent (SGD) algorithm replaces the true gradient by an unbiased estimate computed from a mini-batch of examples:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \cdot \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}_t),$$

where $\mathcal{B} \subset \{1, \ldots, N\}$ is a random mini-batch. This stochastic approximation introduces noise but enables efficient optimisation, especially for large-scale problems.

With these foundations in place, we now turn to the earliest and simplest learnable model: the perceptron.

Probability Spaces and Axioms

Probability theory begins with a mathematical model for random experiments. The fundamental structure is the **probability space**.

Definition 0.1 (Probability Space)**.** *A probability space is a triple* $(\Omega, \mathcal{F}, P)$ *where:*

(i) Ω *is a nonempty set, called the* ***sample space****. Its elements* $\omega \in \Omega$ *are elementary outcomes.*

(ii) $\mathcal{F}$ *is a* σ*-****algebra*** *(or* σ*-field) of subsets of* Ω*: a collection of subsets containing* Ω*, closed under complement and countable union. The elements* $A \in \mathcal{F}$ *are called* ***events****.*

(iii) $P : \mathcal{F} \to [0,1]$ *is a* ***probability measure*** *satisfying the Kolmogorov axioms:*

(a) $P(A) \geq 0$ *for all* $A \in \mathcal{F}$*.*

(b) $P(\Omega) = 1$*.*

(c) If $A_1, A_2, \ldots$ *are pairwise disjoint* ($A_i \cap A_j = \varnothing$ *for* $i \neq j$)*, then*

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \quad \textit{(}\sigma\textit{-additivity).}$$

The σ-algebra $\mathcal{F}$ formalizes which subsets of Ω are considered measurable (i.e., we can assign a probability). In discrete settings we often take $\mathcal{F} = 2^{\Omega}$ (all subsets). For continuous spaces like $\mathbb{R}$, we use the Borel σ-algebra generated by open intervals.

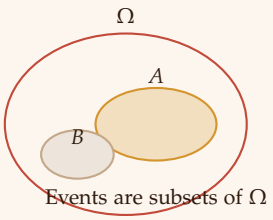


Figure 1: A sample space Ω with two events A and B.

Properties of Probability Measures

From the axioms we derive several useful properties:

- $P(\varnothing) = 0$.
- Finite additivity: for disjoint $A_1, \ldots, A_n$, $P(\bigcup_{i=1}^{n} A_i) = \sum_{i=1}^{n} P(A_i)$.
- Monotonicity: If $A \subseteq B$, then $P(A) \leq P(B)$.
- Complement: $P(A^c) = 1 - P(A)$.
- Union bound (Boole's inequality): $P(\bigcup_i A_i) \leq \sum_i P(A_i)$.
- Inclusion–exclusion: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

Conditional Probability and Independence

Conditional probability quantifies how the probability of an event changes given that another event has occurred.

Definition 0.2 (Conditional Probability)**.** *For events* A *and* B *with* $P(B) > 0$*, the conditional probability of* A *given* B *is*

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Definition 0.3 (Independence). *Two events A and B are **independent** if $P(A \cap B) = P(A)P(B)$. Equivalently, $P(A \mid B) = P(A)$ when $P(B) > 0$. A collection $\{A_i\}$ is mutually independent if for every finite subset I,*

$$P\left(\bigcap_{i \in I} A_i\right) = \prod_{i \in I} P(A_i).$$

Bayes' Theorem A fundamental result that reverses conditioning:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}, \qquad \text{where } P(B) = \sum_i P(B \mid A_i)P(A_i)$$

for a partition $\{A_i\}$ of Ω. Bayes' theorem is the cornerstone of Bayesian inference and permeates modern machine learning (e.g., naive Bayes classifiers, Bayesian neural networks).

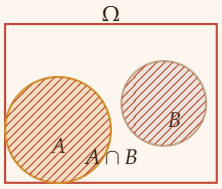


Figure 2: Bayes' theorem relates $P(A|B)$ to $P(B|A)$ via the prior $P(A)$ and evidence $P(B)$.

Random Variables

Often we are not directly interested in elementary outcomes but in numerical quantities derived from them.

Definition 0.4 (Random Variable). *A **random variable** X on a probability space $(\Omega, \mathcal{F}, P)$ is a function $X : \Omega \to \mathbb{R}$ such that for every Borel set $B \subseteq \mathbb{R}$, the preimage $X^{-1}(B) = \{\omega : X(\omega) \in B\}$ belongs to $\mathcal{F}$ (i.e., X is measurable). This condition ensures that probabilities of events involving X are well-defined.*

Discrete and Continuous Random Variables

- **Discrete** X takes values in a countable set $\mathcal{X} \subseteq \mathbb{R}$. It is described by its **probability mass function (PMF)** $p_X(x) = P(X = x)$.
- **Continuous** X has a **probability density function (PDF)** $f_X(x)$ such that for any interval $[a, b]$,

$$P(a \le X \le b) = \int_a^b f_X(x)\,dx.$$

The PDF satisfies $f_X(x) \ge 0$ and $\int_{-\infty}^{\infty} f_X(x)\,dx = 1$.

The **cumulative distribution function (CDF)** $F_X(x) = P(X \le x)$ is defined for any random variable and is non-decreasing, right-continuous, with $\lim_{x \to -\infty} F_X(x) = 0$, $\lim_{x \to \infty} F_X(x) = 1$.

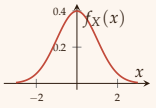


Figure 3: PDF of the standard normal distribution.

Expectation and Moments

The expectation (mean) is the weighted average of a random variable.

Definition 0.5 (Expectation). *For a discrete random variable X with PMF p_X, $\mathbb{E}[X] = \sum_x x\, p_X(x)$ (provided the sum converges absolutely). For continuous X with PDF f_X, $\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, f_X(x)\,dx$.*

Properties of Expectation

- Linearity: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ for constants a, b.
- If $X \geq 0$ almost surely, then $\mathbb{E}[X] \geq 0$.
- Law of the unconscious statistician: $\mathbb{E}[g(X)] = \sum_x g(x)p_X(x)$ (discrete) or $\int g(x)f_X(x)\,dx$ (continuous).

Variance and Covariance

Definition 0.6 (Variance)**.** *The variance of X measures its spread around the mean:*

$$\mathrm{Var}(X) = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

The standard deviation is $\sigma_X = \sqrt{\mathrm{Var}(X)}$.

Definition 0.7 (Covariance and Correlation)**.** *For two random variables X, Y,*

$$\mathrm{Cov}(X, Y) = \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

The correlation coefficient is $\rho(X, Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$, satisfying $-1 \leq \rho \leq 1$.

If X and Y are independent, then $\mathrm{Cov}(X, Y) = 0$ (the converse is not true in general).

Important Distributions for Machine Learning

Several probability distributions appear repeatedly in machine learning.

Bernoulli and Binomial

- **Bernoulli**(θ): $X \in \{0, 1\}$, $P(X = 1) = \theta$, $P(X = 0) = 1 - \theta$. $\mathbb{E}[X] = \theta$, $\mathrm{Var}(X) = \theta(1 - \theta)$. Used for binary classification.
- **Binomial**(n, θ): $X = \sum_{i=1}^{n} Y_i$ with Y_i i.i.d. Bernoulli(θ). PMF: $P(X = k) = \binom{n}{k}\theta^k(1 - \theta)^{n-k}$, $\mathbb{E}[X] = n\theta$, $\mathrm{Var}(X) = n\theta(1 - \theta)$.

Gaussian (Normal) Distribution

The most ubiquitous continuous distribution.

$$\mathcal{N}(\mu, \sigma^2): \quad f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad \mathbb{E}[X] = \mu,\ \mathrm{Var}(X) = \sigma^2.$$

The standard normal $\mathcal{N}(0, 1)$ has mean 0 and variance 1. By the Central Limit Theorem, sums of many independent random variables tend to a Gaussian.

Exponential and Laplace

- **Exponential**(λ): $f(x) = \lambda e^{-\lambda x}$, $x \geq 0$, $\mathbb{E}[X] = 1/\lambda$. Used for waiting times.
- **Laplace**(μ, b): $f(x) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right)$. Heavier tails than Gaussian; used in robust regression.

Multivariate Gaussian

For a random vector $\mathbf{X} = (X_1, \ldots, X_d)^\mathsf{T}$,

$$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}): \quad f(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right),$$

with mean vector $\boldsymbol{\mu} \in \mathbb{R}^d$ and positive-definite covariance matrix $\boldsymbol{\Sigma}$. This is the foundation of many latent variable models and deep generative models (e.g., variational autoencoders).

Limit Theorems

These theorems justify why averaging works and why the Gaussian distribution appears so often.

Theorem 0.1 (Law of Large Numbers (LLN))**.** *Let $X_1, X_2, \ldots$ be i.i.d. with $\mathbb{E}[X_i] = \mu$. Then the sample average $\overline{X}_n = \frac{1}{n}\sum_{i=1}^n X_i$ converges to μ in probability:*

$$\forall \varepsilon > 0, \quad \lim_{n\to\infty} P(|\overline{X}_n - \mu| > \varepsilon) = 0.$$

Theorem 0.2 (Central Limit Theorem (CLT))**.** *Under the same i.i.d. assumption with finite variance σ^2, the scaled sample average converges in distribution to a standard normal:*

$$\sqrt{n}\,\frac{\overline{X}_n - \mu}{\sigma} \xrightarrow{d} \mathcal{N}(0,1).$$

Equivalently, $\overline{X}_n \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n.

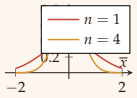


Figure 4: The CLT: as n increases, the distribution of $\overline{X}_n$ becomes narrower and more Gaussian.

The CLT explains why the Gaussian prior is a common choice and why many estimators are asymptotically normal.

Multivariate Random Vectors and Joint Distributions

When dealing with multiple random variables simultaneously, we use the joint distribution.

Definition 0.8 (Joint PDF/PMF)**.** *For a random vector $(X_1, \ldots, X_d)$:*

- *Discrete: joint PMF $p(x_1, \ldots, x_d) = P(X_1 = x_1, \ldots, X_d = x_d)$.*
- *Continuous: joint PDF $f(x_1, \ldots, x_d)$ satisfying $P(\mathbf{X} \in A) = \int_A f(\mathbf{x})\, d\mathbf{x}$.*

Marginal and Conditional Distributions

- **Marginal** PMF/PDF: e.g., $p_{X_1}(x_1) = \sum_{x_2,\dots,x_d} p(x_1,\dots,x_d)$ (discrete) or $f_{X_1}(x_1) = \int \cdots \int f(x_1,\dots,x_d)\,dx_2 \cdots dx_d$ (continuous).
- **Conditional** distribution of X given $Y = y$: $f_{X|Y}(x|y) = \frac{f_{X,Y}(x,y)}{f_Y(y)}$ (for continuous) and similarly for PMF.

Independence of Random Variables

Random variables $X_1,\dots,X_d$ are independent iff their joint distribution factorizes:

$$f(x_1,\dots,x_d) = \prod_{i=1}^{d} f_{X_i}(x_i) \quad \text{(or PMF analog)}.$$

Independence simplifies many computations and is a common modeling assumption (e.g., naive Bayes).

Law of Total Expectation and Variance

For two random variables X, Y,

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X \mid Y]\big], \qquad \operatorname{Var}(X) = \mathbb{E}\big[\operatorname{Var}(X \mid Y)\big] + \operatorname{Var}\big(\mathbb{E}[X \mid Y]\big).$$

These identities are crucial in deriving algorithms like expectation–maximization and in analyzing stochastic gradient descent.

Information Theory for Machine Learning

Information theory provides tools to quantify uncertainty and information content, which are essential for understanding entropy, cross-entropy loss, and Kullback–Leibler divergence.

Definition 0.9 (Entropy)**.** *For a discrete random variable X with PMF p, the **Shannon entropy** is*

$$H(X) = -\sum_x p(x) \log p(x),$$

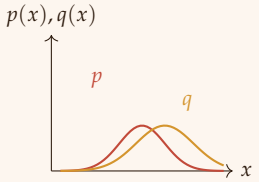
measured in bits if the logarithm is base 2, or nats if natural log. Entropy measures the average uncertainty about X.

Definition 0.10 (Kullback–Leibler Divergence)**.** *For two probability distributions p and q over the same alphabet,*

$$D_{KL}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

(with $0 \log 0 = 0$). D_{KL} is non-negative and zero iff $p = q$. It is asymmetric and measures how much information is lost when q approximates p.

The cross-entropy between p and q is $H(p,q) = H(p) + D_{\mathrm{KL}}(p \parallel q) = -\sum_x p(x) \log q(x)$. Minimizing cross-entropy is equivalent to minimizing KL divergence when p is fixed (the true distribution) – this is exactly what many classification neural networks do (categorical cross-entropy loss).



KL divergence measures the dissimilarity of p and q

Figure 5: KL divergence $D_{\mathrm{KL}}(p \parallel q)$ is zero when the distributions coincide.

Introduction to Artificial Neural Networks

Perceptron

Nearly every modern artificial neural network, from simple classifiers to sprawling deep learning architectures, traces its origins to a simple computational unit called the **perceptron** [1].

Developed by Frank Rosenblatt in 1947, inspired by the work of McCulloch and Pitts in 1943. The perceptron, sometimes called the *sigmoid neuron*, it's traced back from the mathematical interpretation of the biological neurons' way of functioning.

At its core, the perceptron is a binary decision-maker. It takes a set of real-valued inputs, weighs their importance, and makes a crisp, threshold-based choice.

[1] While most neural networks rely on perceptron-like units, alternative paradigms exist, such as Hebbian learning (inspired by synaptic plasticity) and the recently revived Kolmogorov–Arnold Network (KAN), which bypasses traditional neuron structures entirely.

Mathematical Definition Given inputs $x_1, x_2, \ldots, x_n \in \mathbb{R}$, the perceptron's output is defined by:

1. **Weighted sum (affine transformation):**

$$z = w_0 + \sum_{i=1}^{n} w_i x_i, \tag{5}$$

   where $w_i \in \mathbb{R}$ are the *weights* and w_0 is the *bias term*.

2. **Thresholding (activation):**

$$o(\mathbf{x}) = \phi(z) = \begin{cases} 1, & \text{if } z > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

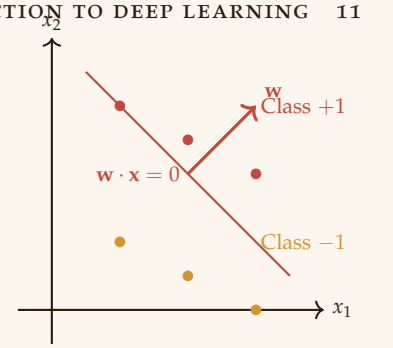
By introducing a dummy input $x_0 = 1$, we can absorb the bias into the weight vector: let $\mathbf{w} = (w_0, w_1, \ldots, w_n)^{\mathsf{T}} \in \mathbb{R}^{n+1}$ and $\mathbf{x} = (1, x_1, \ldots, x_n)^{\mathsf{T}}$. Then $z = \mathbf{w} \cdot \mathbf{x}$ and the output is $o(\mathbf{x}) = \mathbf{1}_{\mathbf{w}\cdot\mathbf{x}>0}$.

Geometric Interpretation The decision boundary is the hyperplane $\{\mathbf{x} \in \mathbb{R}^{n+1} : \mathbf{w} \cdot \mathbf{x} = 0\}$ that separates the input space into two half-spaces. For $n = 2$, this reduces to a line:

$$w_0 + w_1 x_1 + w_2 x_2 = 0,$$

with slope determined by $-w_1/w_2$ and intercept by $-w_0/w_2$. The weight vector $\mathbf{w}$ is orthogonal to this decision boundary and points towards the region where $\mathbf{w} \cdot \mathbf{x} > 0$.

The perceptron can only represent *linearly separable* functions. The classic XOR problem (Figure 6) is not linearly separable, which motivated the development of multilayer architectures.

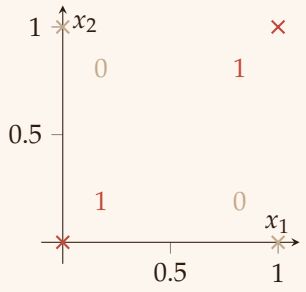


Decision boundary: $w_0 + w_1 x_1 + w_2 x_2 = 0$

Figure 6: XOR operation: no single straight line can separate the two classes.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
1	0	1
0	1	1
1	1	0

(a) XOR truth table



(b) XOR as a non-linear classification problem

Gradient Descent and Stochastic Gradient Descent

Perceptron Training Rule Given a training set $\{(\mathbf{x}_d, t_d)\}_{d=1}^N$, the perceptron learning rule updates weights whenever a misclassification occurs:

$$w_i \leftarrow w_i + \Delta w_i, \qquad \Delta w_i = \eta\,(t_d - o_d)\,x_{i,d},$$

where $\eta > 0$ is the learning rate, t_d is the target (0 or 1), o_d the perceptron's output, and $x_{i,d}$ the i-th component of $\mathbf{x}_d$. This rule converges if the data are linearly separable; otherwise it may oscillate.

Delta Rule and Linear Units For non-separable problems, we consider a linear unit without threshold: $o(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x}$. Define the sum of squared errors over the training set:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{d=1}^{N} (t_d - o_d)^2.$$

The gradient of E with respect to $\mathbf{w}$ is

$$\nabla_{\mathbf{w}} E(\mathbf{w}) = \sum_{d=1}^{N} (o_d - t_d)\,\mathbf{x}_d.$$

Thus the gradient descent update is

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} E(\mathbf{w}) \quad \Longrightarrow \quad \Delta w_i = \eta \sum_{d=1}^{N} (t_d - o_d)\,x_{i,d}.$$

Stochastic Gradient Descent (SGD) For large N, computing the full gradient is expensive. SGD approximates it using a single random example (or a mini-batch):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\,(o_d - t_d)\,\mathbf{x}_d, \qquad d \sim \text{Uniform}\{1, \ldots, N\}.$$

The noisy updates allow the algorithm to escape shallow local minima and are the basis for training deep networks.

Multilayer Perceptron and Backpropagation

A multilayer perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. Each neuron uses a differentiable nonlinear activation function $f(\cdot)$ (e.g., logistic sigmoid $\sigma(z) = 1/(1+e^{-z})$, hyperbolic tangent $\tanh(z)$, or ReLU $\max(0, z)$). The universal approximation theorem (Section) guarantees that MLPs with at least one hidden layer can approximate any continuous function on a compact set to arbitrary accuracy, provided enough hidden units.

Backpropagation Algorithm Let the network have L layers, with weights $w_{ij}^{(\ell)}$ connecting unit i in layer $\ell - 1$ to unit j in layer ℓ. For an input $\mathbf{x}$, forward propagation computes:

$$a_j^{(\ell)} = \sum_i w_{ij}^{(\ell)} o_i^{(\ell-1)}, \qquad o_j^{(\ell)} = f(a_j^{(\ell)}),$$

where $o_i^{(0)} = x_i$. The output layer produces $\hat{y}_k = o_k^{(L)}$. The loss for a single example with target $\mathbf{t}$ is, e.g., half the squared error:

$$E = \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - \hat{y}_k)^2.$$

Backpropagation computes the gradient $\partial E / \partial w_{ij}^{(\ell)}$ via the chain rule. Define the error signal for output units:

$$\delta_k^{(L)} = \frac{\partial E}{\partial a_k^{(L)}} = (t_k - \hat{y}_k)\, f'(a_k^{(L)}).$$

For a hidden layer ℓ, the error is back-propagated:

$$\delta_j^{(\ell)} = f'(a_j^{(\ell)}) \sum_k w_{jk}^{(\ell+1)} \delta_k^{(\ell+1)}.$$

The gradient is then

$$\frac{\partial E}{\partial w_{ij}^{(\ell)}} = \delta_j^{(\ell)}\, o_i^{(\ell-1)}.$$

Finally, weights are updated using SGD:

$$w_{ij}^{(\ell)} \leftarrow w_{ij}^{(\ell)} - \eta\, \delta_j^{(\ell)} o_i^{(\ell-1)}.$$

Algorithm 1: Backpropagation (online version)

Initialize all weights $w_{ij}^{(\ell)}$ randomly.
for each training example $(\mathbf{x}, \mathbf{t})$ **do**
  Forward pass: compute all $a_j^{(\ell)}$ and $o_j^{(\ell)}$.
  Output errors: $\delta_k^{(L)} = (t_k - o_k^{(L)}) f'(a_k^{(L)})$.
  for $\ell = L-1$ down to 1 **do**
    $\delta_j^{(\ell)} = f'(a_j^{(\ell)}) \sum_k w_{jk}^{(\ell+1)} \delta_k^{(\ell+1)}$.
  end for
  for each weight $w_{ij}^{(\ell)}$ **do**
    $w_{ij}^{(\ell)} \leftarrow w_{ij}^{(\ell)} + \eta\, \delta_j^{(\ell)} o_i^{(\ell-1)}$.
  end for
end for

Universal Approximation Theorem

One of the most important theoretical results for neural networks is the universal approximation theorem, originally proved by Cybenko (1989) for sigmoidal activation functions. The theorem states:

Theorem 0.1 (Cybenko, 1989). *Let $f : \mathbb{R}^n \to \mathbb{R}$ be any continuous function on the n-dimensional hypercube $[0,1]^n$. Then for any $\varepsilon > 0$, there exists a single-hidden-layer feedforward network with sigmoidal activation functions and a finite number of hidden units such that the network output $\hat{f}(\mathbf{x})$ satisfies*

$$\sup_{\mathbf{x} \in [0,1]^n} \left| \hat{f}(\mathbf{x}) - f(\mathbf{x}) \right| < \varepsilon.$$

The theorem implies that even a shallow network (one hidden layer) is *dense* in the space of continuous functions when the number of hidden units is allowed to grow. However, it does not guarantee that such a network can be learned efficiently (the number of units may be astronomically large). Modern deep learning uses *depth* to represent complex functions more compactly.

Figure 7 illustrates the idea: a piecewise linear approximation of $\sin(x)$ improves as the number of linear regions (analogous to hidden units) increases.

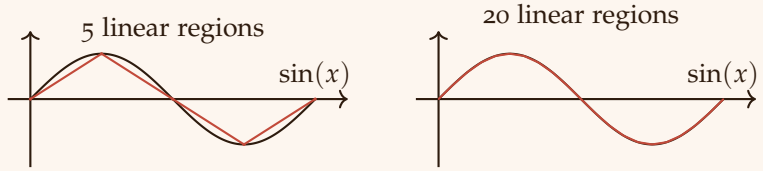


Figure 7: Approximating $\sin(x)$ by piecewise linear functions: more linear regions (more hidden units) yield a better approximation. This illustrates the essence of the universal approximation theorem.

References

[1] V. Vapnik, *Statistical Learning Theory*. Wiley-Interscience, 1998.

[2] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.